

Accumulation of Tallied Data

1 Introduction

The estimators in DEGAS 2 are calculated by the routines in `score.web`, which are invoked during tracking. At the end of each flight, the resulting data are accumulated into the global arrays described in the statistics class (package abbreviation `sa`) via the subroutines in `stat.web`.

The data accumulation process takes place over a range of scales to optimize performance on massively parallel platforms. The basic breakdown is:

flt for “flight”. This always refers to a single flight.

frag for “fragment”. To facilitate load balancing, only a fraction (or “fragment”) of the total number of flights per processor are sent at a time to each process. As each process completes, an additional fragment is sent until all have been completed.

fin for “final”. In the simplest case, this refers to the global accumulation of data from all processes. Again, in the simplest case, the data are actually accumulated directly in an array from the output class.

This process is complicated by two additional optimizations. First, as is documented in the statistics class, these arrays may be compressed, especially at the `flt` level, to reduce the amount of data transmitted. Second, in runs on larger machines (say, > 100 cores or processes) fragments are accumulated into intermediate “final” arrays over groups of processors to reduce the number of accumulations that must be performed by the master process, avoiding network transmission collisions. The machinery facilitating this is contained in `degasinit.web`.

2 Basic Expressions

The sources of neutral particles to be simulated in a given DEGAS 2 run are broken up into “groups” according to their physical nature, e.g., “plate recycling”, “gas puff”, etc., as described in the sources class. Because the number of flights and total particle current associated with each source group are specified independently by the user, the data for each group are accumulated separately and are available by themselves within the output arrays.

The fundamental expressions used in the data accumulation process are, thus, for a specific source group i . The other principal indexing parameters for the output arrays are the geometric zones, z , and the tallies, as contained in the tally class. To minimize confusion, we do not introduce any explicit index for the latter; the following expressions apply independently for each tally. The first accumulated moment, to be associated with the mean in the end is:

$$S_1^i(z) = \sum_{j=1}^{N_i} \rho_j \left(\sum_{k \in z} \xi_{j,k} \right) / \sum_{j=1}^{N_i} \rho_j, \quad (1)$$

where: i is the source group index, the “1” refers to the first statistical moment, j runs over all of the N_i flights in group i , ρ_j is the relative statistical weight of flight j , k is the list of events scored by flight j (e.g., collisions) in zone z , and $\xi_{j,k}$ is the estimator value (see below) for this tally recorded by flight j during event k . The statistical weight $\rho_j \equiv 1$ in most cases, but can be set otherwise by the user to effect importance sampling via the input arrays `source_segment_relwt`. Note that this quantity

is *fixed* for a given flight (indeed, is the same for all flights launched from that source segment) and is distinct from the statistical weight `[pt_w(fl_current(x))` in subroutine `follow`], designated by w in the estimator expressions below, which may vary during flight tracking, but *always* has an initial value of $w = 1$.

Estimators are discussed in greater generality in the User's Manual. Here, we write out the ones of greatest interest to provide adequate context for the statistical moment expressions. First, the estimator most commonly used in compiling test particle data is the track length estimator,

$$\xi_g^{\text{TLE}} = w \frac{1 - \exp(-\sigma_i t_V)}{\sigma_i} g, \quad (2)$$

where t_V is the time the particle track spent in volume V , w = weight at start of the time step. Note that in the limit $\sigma_i t_V \ll 1$, $[1 - \exp(-\sigma_i t_V)]/\sigma_i \rightarrow t_V$, so that estimator scales directly with the time spent in a zone. For longer time intervals, e.g., as might be the case when other reaction rates are small, the exponential factor effectively accounts for weight reduction along the track. The function g can be a test particle attribute, such as mass, momentum, energy, or reaction-related, e.g., ion momentum source due to charge exchange. Volumetric plasma sources, such as the latter, enter the track-length estimator as averages over the background distribution.

The second type of estimator is the collision estimator,

$$\xi_g^{\text{CE}} = \sum_{m_s} w_{m_s} \frac{g}{\sigma_s}, \quad (3)$$

where the sum is over m_s , all scattering collisions within V , and is equivalent to the $\sum_k$ in Eq. (1). The quantity w_{m_s} is the weight at m_s th collision. If suppressed absorption is being used, as is normally the case, the particle weight will decrease between the collisions. Again, g is a test particle attribute.

There is an analogous expression for the collision estimator associated with a particular reaction,

$$\xi_g^{\text{CE}} = \sum_{m_j} w_{m_j} \frac{g}{\sigma_j}, \quad (4)$$

where the sum is now over m_j collisions of reaction j within V . The function g is some quantity related to the reaction and is likely something known only at collisions. One example of such a quantity would be the energy source due to H_2 dissociation (i.e., the energy exchange is determined only once the product velocities have been determined).

The equivalent expression for the second moment is:

$$S_2^i(z) = \sum_{j=1}^{N_i} \rho_j \left(\sum_{k \in z} \xi_{j,k} \right)^2 / \sum_{j=1}^{N_i} \rho_j - \left[\sum_{j=1}^{N_i} \rho_j \left(\sum_{k \in z} \xi_{j,k} \right) / \sum_{j=1}^{N_i} \rho_j \right]^2. \quad (5)$$

While this expression is very close to that for the variance appearing in statistics textbooks, it is not suitable for the multi-level data accumulation used by DEGAS 2. How one would break up the sum in the numerator of Eq. (1) into smaller sets (fragments) of flights j is clear. To do likewise for the second moment, we need to instead work with:

$$\sum_{j=1}^{N_i} \rho_j \left(\sum_{k \in z} \xi_{j,k} \right)^2 / \sum_{j=1}^{N_i} \rho_j = S_2^i(z) + \left[\sum_{j=1}^{N_i} \rho_j \left(\sum_{k \in z} \xi_{j,k} \right) / \sum_{j=1}^{N_i} \rho_j \right]^2. \quad (6)$$

3 Accumulating Data

We now motivate the specific expressions used to accumulate data from an individual flight into a fragment, and a fragment into a final total. In the interest of clarity, we introduce some simplified notation: $\sum_{j=1}^{N_i} \rho_j \rightarrow w$, $S_1^i(z) \rightarrow S_1$, and $S_2^i(z) \rightarrow S_2$. The subscript i in the following represents “incremental” data (e.g., an individual flight), b represents the “base” data (e.g., the fragment containing data from earlier flights); the result of the accumulation will be new base data, indicated with a prime. The familiar form for the first moment is:

$$S'_{1b} = (w_i S_{1i} + w_b S_{1b}) / (w_i + w_b), \quad (7)$$

with the new base weight being:

$$w'_b = w_i + w_b. \quad (8)$$

But, the code actually uses an equivalent form to maximize numerical accuracy:

$$S'_{1b} = S_{1b} + (S_{1i} - S_{1b}) \frac{w_i}{w_i + w_b}. \quad (9)$$

The analogous expression following from Eq. (6) is:

$$(S'_{2b} + S'^2_{1b})w'_b = (S_{2b} + S^2_{1b})w_b + (S_{2i} + S^2_{1i})w_i. \quad (10)$$

Thus,

$$S'_{2b} = [(S_{2b} + S^2_{1b})w_b + (S_{2i} + S^2_{1i})w_i - S'^2_{1b}w'_b] / w'_b. \quad (11)$$

The equivalent form that is coded up in `stat.web` is:

$$S'_{2b} = [S_{2b}w_b + S_{2i}w_i + (S_{1i} - S_{1b})(S_{1i} - S'_{1b})w_i] / w'_b. \quad (12)$$

The equivalence of Eqs. (11) and (12) can be demonstrated most readily by adding and subtracting $S_{1b}(S_{1i} - S'_{1b})w_i$ inside the square brackets of Eq. (11) and using Eq. (7) for S'_{1b} . Again, this form improves numerical accuracy, e.g., in the case where $S_{1i} \simeq S_{1b}$.

To see this, consider two limiting cases. First, look at the one in which the increment is much smaller than the base value: $S_{1i} = \epsilon S_{1b}$, where $\epsilon \ll 1$. We insert this in Eqs. (11) and (12), and drop second order terms in ϵ relative to higher order terms (as if lost due to finite precision). Note that we expect $w_i \leq w_b$ in almost all cases. since the S_2 terms are the same in these equations, we work only with the S_1 terms. So, the familiar arrangement yields:

$$S^2_{1b}w_b + S^2_{1i}w_i - S'^2_{1b}w'_b \quad (13)$$

$$\rightarrow S^2_{1b}(w_b + \epsilon^2 w_i) - S^2_{1b}(w_b + \epsilon w_i)^2 / (w_b + w_i) \quad (14)$$

$$= S^2_{1b}[w_b + \epsilon^2 w_i - \frac{w_b^2}{w_i + w_b}(1 + \epsilon \frac{w_i}{w_b})^2] \quad (15)$$

$$\simeq S^2_{1b}[w_b - \frac{w_b^2}{w_i + w_b}(1 + 2\epsilon \frac{w_i}{w_b})] \quad (16)$$

$$= S^2_{1b} \frac{w_i w_b}{w_i + w_b} (1 - 2\epsilon). \quad (17)$$

The equivalent terms from Eq. (12):

$$(S_{1i} - S_{1b})(S_{1i} - S'_{1b})w_i \quad (18)$$

$$\rightarrow (\epsilon - 1)S_{1b}[\epsilon S_{1b} - S_{1b} \frac{w_b}{w_i + w_b}(1 + \epsilon \frac{w_i}{w_b})] \quad (19)$$

$$= S^2_{1b} \frac{w_i w_b}{w_i + w_b} (\epsilon - 1)^2 \quad (20)$$

$$\simeq S^2_{1b} \frac{w_i w_b}{w_i + w_b} (1 - 2\epsilon), \quad (21)$$

i.e., the same as Eq. (17).

However, if $S_{1i} = (1 + \epsilon)S_{1b}$,

$$S_{1b}^2 w_b + S_{1i}^2 w_i - S_{1b}'^2 w_b' \quad (22)$$

$$\rightarrow S_{1b}^2 [w_b + (1 + \epsilon)^2 w_i] - S_{1b}^2 (w_i + w_b) \left(1 + \frac{\epsilon w_i}{w_i + w_b}\right)^2 \quad (23)$$

$$\simeq S_{1b}^2 [w_b + (1 + 2\epsilon)w_i - (w_i + w_b) \left(1 + \frac{2\epsilon w_i}{w_i + w_b}\right)] \quad (24)$$

$$\rightarrow 0! \quad (25)$$

But, with the modified form,

$$(S_{1i} - S_{1b})(S_{1i} - S_{1b}')w_i \quad (26)$$

$$\rightarrow \epsilon S_{1b} [(1 + \epsilon)S_{1b} - S_{1b} \left(1 + \frac{\epsilon w_i}{w_i + w_b}\right)] w_i \quad (27)$$

$$= \epsilon^2 S_{1b}^2 w_b w_i. \quad (28)$$

That is, because $S_{1i} - S_{1b}$ appears in both Eqs. (9) and (12), the factors of ϵ enter via separate terms, reducing round-off errors.

4 Implementation in the Code

The flexibility and generality of the scoring machinery in DEGAS 2 can make relating the expressions in the previous sections to the lines in the code challenging. Virtually all of the scoring in DEGAS 2 begins in the tracking loop of subroutine `follow` with a call to one of: `score_sources`, `score_reaction`, `score_test`, or `score_sector` (which invokes `score_diagnostics`). Note that `score_reaction` and `score_sector` also serve as the starting point for processing the corresponding action indicated by its name. This was also true of `score_sources` until the option of sampling on the master processor was implemented.

For all of the scoring routines utilizing multiple estimator types (i.e., all except `score_sector` / `score_diagnostics`), the first argument is a macro constant indicating the type of estimator to be used for that call. Note that `score_sources` is also called with the post-processing estimator in subroutine `post_process_source_scores`.

The routines `score_test` and `score_reaction` compute the estimators described in Sec. 2. The second argument for these represents the time-like quantity in Eqs. (2) - (4). In particular, for the calls using the track length estimator, `tl_est_track`, the variable `t_fac` is

$$t_{fac} = \frac{1 - \exp(-\sigma_i t_V)}{\sigma_i}. \quad (29)$$

For the calls with the collision estimator, `tl_est_collision`, the second argument is either the inverse of the total scattering rate, σ_s (`other_rate` in the code), as in Eq. (3), or the rate of a single process, σ_j (`rate[i]` in the code), as in Eq. (4).

All of the argument lists for the scoring routines contain an entry representing the current flight or, in the case of `score_sector`, the entire flight stack. In the latter case, the processing of the products of plasma-material interactions (PMI) is handled in `score_sector`, altering the flight stack. This is analogous what is done by the collision estimator call to `score_reaction` except that it returns the list of products (and reagents) in the particle array, `prod`, which is explicitly processed in subroutine `follow` into modifications of the flight stack, `fl_pointer(x)` and `fl_current(x)`. In terms of scoring, the current flight provides the weight factor, w , in Eqs. (2) - (4).

All of the scoring routines also require the `estimator_factors` argument, a 1-D array with an entry for each of the tallies. This dynamic array is actually allocated in subroutine `do_flights_master` so that it can also be used in the post-process scoring routines as well as in the scoring routines called by subroutine `follow`. In the simplest cases of `score_sources` and `score_diagnostics` (called by `score_sector`), the nominal content of each entry in `estimator_factors` is the weight of the current particle. In `score_test` and `score_reaction`, this is multiplied by the time-like quantity discussed above and represented locally by the variable `est_fac`.

The `estimator_factors` array also enforces the estimator dependence of each tally contained in the `tally_est_test` and `tally_est_reaction` arrays for test and reaction / source scores, respectively. These arrays provide an overall 1 (0) factor if the current estimator is (is not) being used for a particular test tally or reaction / source tally for a particular reaction or source type (see the tally class). If all of the `estimator_factors` for the current score type are zero, the local flag `need_scores` will be set to false and the remainder of the scoring process will be bypassed.

Assuming that is not the case (i.e., `need_scores` is true), the quantities corresponding to g in Eqs. (2) - (4) are computed; these are stored in the local array `scoring_data`. The length of this array is the macro constant `scoring_data_max`, defined as part of the tally class in `tally.hweb`. The nominal content of `scoring_data` is the set of dependent variables contained in the problem class, i.e., the macro parameters `pr_var_mass`, etc. A key feature of this list is that it is extended as a part of `problemsetup`, e.g., to handle user-specified emission lines, with the total number of entries being given by `pr_var0_num`. During `tallysetup`, the assertion that `scoring_data_max > pr_var0_num` is checked. The specific reason that the length of `scoring_data` is not set to `pr_var0_num` instead has been forgotten!

Subroutine `test_score` is the simplest of the scoring routines. The principal values of the variable g , computed in subroutine `test_scoring_data`, are the particle's mass, momentum vector, and energy. The `pr_var_xy_stress` variable, mv_1v_2 , is also provided for use in the Couette Flow example. Note that additional elements of the stress tensor could be compiled in the same manner, if needed for a future application.

In subroutine `score_reaction`, the task of filling the `scoring_data` array is handled by the "collision" routines. When a collision type estimator is being processed, an actual collision routine, which results in modifications to the flight stack, is called via subroutine `pick_reaction`. For track and post-processing estimators, the track-length versions of the reaction routines are invoked via subroutine `pick_track_reaction`. These have no impact on the characteristics of the current flight; in fact, subroutine `follow` asserts that these calls to `score_reaction` return no products. All of these reaction routines provide the entries in `scoring_data` corresponding to the changes in "problem species" (the union of the test and background species lists) mass, momentum vector, and energy. For those processes that result in the emission of photons, e.g., ionization and dissociative excitation, values for the emission rate and velocity vector of the emitting species are included in the `scoring_data` array.

Each of the source types has a corresponding "data" routine, e.g., subroutine `puff_data`, that is called from subroutine `score_sources` and fills in the `scoring_data` mass, momentum vector, and energy sources for it. Subroutine `plate_data` handles both the `so_plate` and `so_plt_e_bins` types. A recombination source is handled more like a reaction, however, in that it provides scoring data for multiple species and also results in light emission. To underscore this point, the `recombination_data` subroutine is actually kept in the `collide.web` file.

4.1 Sector and Diagnostic Scoring

The scoring of sectors and diagnostics in `score_sector` is complicated first by the presence of two sectors at a zone boundary and second by the need to track mass, momentum, and energy both into and out of the sectors. The subroutine first sets `sector1` to `lc_sector` and `sector2` to `lc_sector_next` for the current flight. Their values will be used to determine if the current flight is at an exit, in which case the current flight is terminated immediately. Or if it is at a wall / target interface, in which case a PMI is processed. The comment after the setting of `sector1` and `sector2` described the objectives of the next three lines, but understanding them is crucial to comprehending the subsequent subroutine calls:

```
nprod = 0
```

Anticipates the possibility of a purely diagnostic interface in which no PMI or exit is processed; i.e., the current flight goes on its way as is.

```
pt_copy(fl_current(x), prod[0])
```

Both the `process_pmi` and `score_diagnostic` subroutines operate on `prod[0]` (and subsequent elements in the case of a PMI), so it must be a copy of the current flight particle.

```
pt_thru_face(fl_current(x))
```

Again, this line anticipates the possibility of a purely diagnostic interface. The macro invokes `lc_thru_face` in `location.hweb`, which in turn sets `lc_cell` to `lc_cell_next` and likewise for `lc_sector` and `lc_sector_next`. As implied by its name, the macro effectively puts the particle on the other side of the interface. If there is a PMI or the current flight is at an exit, the flight is killed, and this macro does not matter. This has no impact on the scoring since that is now going to be done on `prod[0]`.

These lines are followed by a loop over the non-diagnostic sectors; i.e., `type = sc_vacuum = 1 → sc_exit = 5`. The way this loop is written amounts to “overkill” in that both “sector” and “sector_next” are tested and that only three of the five sector types (wall, target, and exit) result in any action. This structure harks back to earlier incarnations of the “sector” concept. In the wall and target cases, the requisite PMI is processed.

The operation of PMI related routines is discussed elsewhere (e.g., in the User’s Manual sections on Recycling and Adding Reactions and PMI), but noting a few details here will help clarify the operation of sectors and diagnostics. Note, first, that the PMI yields may be functions of the test particle incident angle, θ , measured relative to the surface normal, and $\cos(\theta)$. The latter is provided by the function `intersection_direction`, which in turn invokes `surface_intersection_direction`. As is noted there, $\cos(\theta) > 0$ means the particle is leaving that surface and cell.

The velocity of the PMI product is returned by the PMI routines as if the particle were impinging on the $X - Y$ plane. When these routines return the product particles to subroutine `pick_pmi`, the outgoing velocity vector is rotated into the proper local orientation by subroutine `surface_reflect`. This is then loaded into the velocity of the `prod` array of each product along with the location of the incoming product. However, this only amounts to copying its location in configuration space, $\vec{x}$, since the next lines of code proceed to set the `lc_cell` to the products to the incoming `lc_cell_next` and vice versa; the same is done for the sector and zone values. This swap is explained in the comment: the scoring is performed as if the incoming (outgoing) particle is leaving (entering) the plasma or vacuum zone and entering (leaving) the solid zone. Once control returns to subroutine `score_sector`, `score_diagnostics` is called. Before delving into it, note that upon its return, the `pt_thru_face`

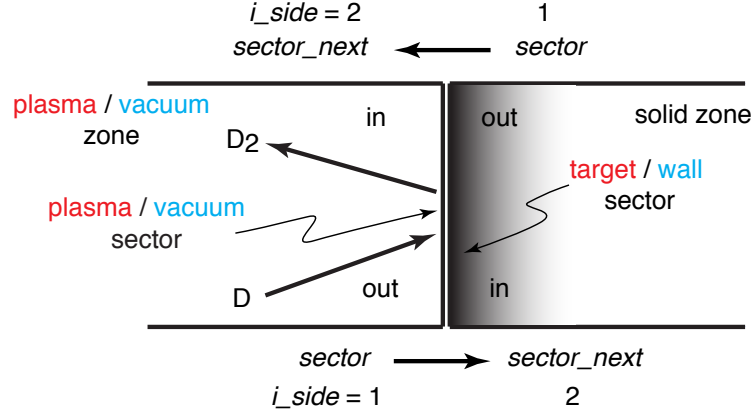


Figure 1: Schematic for a desorption PMI showing the sector values used by subroutine `score_diagnostics`.

macro is called for each product so that each is then labeled as being in a plasma or vacuum zone (and cell) and is, thus, ready to be tracked. The face and sector values at this point are incorrect, but these are not needed for tracking and test or reaction scoring.

The values of the `sector` and `sector_next` for the current and product particles when `score_diagnostics` is called are shown schematically in Fig. 1. An outer loop, $j = 0 \rightarrow n_{\text{prod}}$, cycles through each of the current and product particles. Inside of this is a loop over all of the tallies of type `sector`, with tally number `jscore`. Yet another loop with index `i_side` considers the two sides of the interface; as in Fig. 1, the values of the local variable `sector` is set using either `lc_sector` or `lc_sector_next`, likewise for the local variable `zone`.

The surface identifier corresponding to `sector` is obtained from the `sector_surface` array; this is unique. The number of sectors, `count`, associated with this surface then comes from `surface_sectors`. The value of `count` may be > 1 for two reasons. First, in 3-D cases a single surface may bound all of the zones constructed from a discretization in the toroidal direction. However, only one of those will have the same zone as the one being scored. More relevant in this case is that purely diagnostic sectors may have been defined for this surface and zone combination; see the description of auxiliary sectors in `definegeometry2d` (note that much of the above description of sectors, etc. is echoed in its preamble and other documentation).

Because of this latter possibility, yet another loop goes through each index of the loop looking at one of these sectors, sector number `subsector`. The key point of this set of nested loops is to identify which tallies need to be updated at the particle's location. For this to be the case, three conditions must be satisfied:

1. The “geometry operator” for the tally `jscore` (e.g., “Wall and Target Counts”; see the `tallysetup` documentation) must include one of the sectors `subsector` associated with `sector`.
2. The zone associated with `subsector` is the same as the current zone value.
3. The dependent variable for tally `jscore` is consistent with the “in” / “out” convention depicted in Fig. 1.

If these conditions are met, `estimator_factors[jscore]` is set to the particle weight; if not, the value is left at the default value of zero. The last of these three is a key point; this is where

the directionality of the diagnostics is enforced. In principle, this could have been done (and in earlier versions of the code, was done) in the subsequent subroutines `sector_scoring_data` or `handle_diagnostics`.

At the completion of these loops, but still inside the loop over the current and PMI product particles, `sector_scoring_data` is called to fill in the `scoring_data` entries for angle, energy, as well as the mass, momentum vector, and energy in both the “in” and “out” directions. These are actually the same since, again, the directionality is enforced in subroutine `score_diagnostics`.

4.2 Subroutine `add_scores`

At the end of each of the main scoring routines, such as `score_test`, is a call to subroutine `add_scores`. Each invocation of it is, thus, for a particular type of tally: `sector`, `test`, or `reaction`; this is the first argument for `add_scores`. The second argument is a particle; this will be the current particle in the case of `score_test`, `score_reaction`, and `score_sources`. In `score_diagnostics`, `add_scores` is called separately for each PMI product (if any; purely diagnostic sectors have no products with `nprod = 0`). The third argument is the problem reaction number; this is non-trivial only for the calls by `score_reaction` and `score_sources`.

The main loop in the subroutine is over all tallies, `jscore`, of the type specified by the first argument. The first task performed in this loop is a check to see if “track conversions” (see tally class documentation and `tallysetup`) are needed for `jscore`. Next comes a test to see whether or not the dependent variable for the score `jscore` is one of the mass, momentum, or energy change variables (from a “collision” reaction or a PMI sector). If it is, the start and end points for a loop over problem species are defined, otherwise, these are set to trivial values. In both cases, the `check_data` variable is compiled from the corresponding entries in the `scoring_data` array to determine whether or not its contents are trivial. If the latter or if the `estimator_factors` value for `jscore` is zero, the current loop is exited, potentially saving significant effort.

Should both the `estimator_factors` and `check_data` values be non-trivial, the geometric aspects of the score are sorted out by subroutine `handle_geometry` (see tally class documentation). For volume tallies, the value of `number_segments` is 1 on return and the content of the `segments` array is just the zone number of the current particle. For reaction (sector) tallies, `handle_detectors` (`handle_diagnostics`) is invoked to fill in these data, as well as any associated “binned” data.

For detectors, the list of “segments” consists of the detector chords associated with the tally `jscore` (its “geometry operator” or “geometry pointer”) that have non-zero contributions from the current zone. If the emission spectrum is being computed for the tally, the “bins” contain that information. Additional details on this can be found elsewhere.

The analogous steps for the sector based diagnostics are taken in `handle_diagnostics`. The structure of this subroutine resembles that of `score_diagnostics` in that it again examines the sectors on the two sides of the current interface. As noted above, the number of sectors to be scored on one of those sides may be > 1 due to the specification of a non-default diagnostic surface there. If no non-default diagnostic surfaces have been defined, the returned value of `number_segments = 1`. Moreover, the associated sector will be part of either a solid or exit zone since that’s the case for all default sectors. One difference between `score_diagnostics` and `handle_diagnostics` is that the latter is focused on a single diagnostic group, like “Wall and Target Counts”; the list of “segments” returned is of members of that group.

Following the call to `handle_geometry`, `add_scores` loops over these segments and their associated “bins”, if any, and sets a corresponding entry in the `index_parameters` array. A third loop goes through all of the “problem species”, if needed for this score, and setting the entry for each in `index_parameters`; if not needed, the loop is trivial. One final loop then goes through

each of independent variables for the score `jscore` and sets its entry in `ind_val` from the contents of `index_parameters`. Once these are all set, and, if needed, “track conversions” applied, `set_comp_scores` is finally called to update the local copy of the output arrays.

4.3 Subroutine `set_comp_scores`

The subroutine itself is relatively simple. Only two arguments are needed: the new contribution to the current score and its location in the output array. The latter is provided by the `out_array_index` macro (see output class documentation), which is constructed using `tally_base`, `tally_dep_var_dim`, and `tally_tab_index`. The new contribution, `datum`, is the relevant entry in the `scoring_data` array multiplied by the `geom_mult` value returned by `handle_geometry` (generally is = 1 except for detector based scores) and the value of `estimator_factors` for `jscore`. The seemingly complex sequence of macros needed to identify the entry in `scoring_data` handles the dependence on problem species and on the rank of the dependent variable (for vector quantities, such as “momentum change”).

For the case in which `stat_comp_flg` is false, `set_comp_scores`, updates the relevant entry in the `stat_flg` array and returns. If compression at the flight level is turned on, the default, the subroutine first checks to see if an entry in the `ifull` slot of the output arrays has been made yet. If so, that entry is updated using the `stat_ptr2short_flg` mapping (see the `stat` class documentation). If not, the size of `stat_flg` array is incremented, the array reallocated if needed, and the pointer arrays `stat_ptr2short_flg` and `stat_ptr2full_flg` assigned.

The end result of all of the calls to `score_test`, etc. for this flight as it is tracked through subroutine `follow` is effectively the quantity S_1 discussed in Sec. 3.

4.4 Post Processed Scores

For reaction type tallies, the user has the option of scoring specific reactions after all of the flights have been run and all of the data have been accumulated. This process is initiated in a section of code near the end of subroutine `do_flights_master` (in `do_flights.web`). The code first checks to see if any of the reaction type tallies use the “post processing” estimator, `tl_est_post_process`; if so, the logical variable `need_scores` is set to true. This is followed by a loop over source groups. The “post” and “output” final conversions (see tally class documentation and `tallysetup` documentation) are applied in this loop; these are performed even if `need_scores` is false.

If `need_scores` is true, the subroutines `post_process_test_scores` and `post_process_source_scores` are called for each source group. The only arguments to the subroutines are the source group number and the same `estimator_factors` array introduced earlier in Sec. 4.

The basic idea behind `post_process_test_scores` is that we create a fake particle (i.e., an instance of the particle class) having the location of each zone in the problem space and weight equal to the number of particles in that zone accumulated during the main run. Because the plasma parameters are constant throughout that zone, all of the reaction rates are the same for these particles. For some scores, such as ionization and light emission rates, the resulting tallies will be the same as those obtained with a track length estimator.

In the original implementation, the “divide number” conversion was used to transform the neutral flux into an average particle velocity that could be used in this same way to obtain post-processed tallies of momentum sources. However, that conversion was found to give incorrect results in runs with multiple source groups. For this reason, an arbitrary velocity vector is used, rendering these post-process tallies incorrect. The same is true for energy source tallies. Note that even if “divide number” worked as intended, additional code would be required to correctly post-process energy source tallies

since the particle class does not have an “energy” property that could be used to transmit the average particle energy in the cell to the reaction scoring routines.

Post-processing is nonetheless useful for some applications. One example is the “fixed neutral density” approach to computed a synthetic GPI diagnostic. That fixed neutral density is obtained via a normal run of the code, using a track-length estimator with a particular plasma background. Subsequent plasma time steps are then run as restarts of DEGAS 2 based on the output file from that initial run, but having the same number of flights. Since the number of flights is unchanged, the main loop of the code does nothing. By changing the scoring of the reactions for the light emission related tallies from track-length or collision to post-process, the light emission rates are re-evaluated with the new plasma parameters, effectively using the same neutral density. These restart simulations take vastly much less time than the original.

The actual implementation in `post_process_test_scores` is, in hindsight, needlessly general. For example, the particle number (and neutral flux vector) tally has had only zone number and test species as the independent variables in virtually every known run of the code so that the explicit loops over all five (`tl_rank_max`) possible independent variables is overkill. The only other less-than-obvious aspect of the code is indicated by the existing comment noting that using the `lc_set_a` macro to set the particle location is potentially time consuming and unnecessary.

The post-processing of source scores is more straightforward in that they are based on the underlying kinetic distributions. Note that a post-processing option has not yet been set up for snapshot sources. Doing so is possible, in principle, and may be advantageous in that the resulting sampling of the snapshot distribution might be more thorough (and, thus, more accurate) than that obtained at run time. Subroutine `post_process_source_scores` loops over all segments, `seg`, of the current source group. The corresponding `kseg = seg - 1` and `xseg` (pointer to a geometry element, such as a sector or zone) parameters are, thus, easily obtained from the segment number. Just as with a post-processed test score, a fake particle is defined for the scoring process with weight equal to the source current at segment `seg`. The fake particle’s location is intended to approximate the average obtained by `set_source_x` in run time sampling. For surface sources, the midpoint of the line segment is used; for volume sources, it’s the zone center. The fake particle’s velocity is non-zero only in the case of a recombination source. In the other cases, the source sampling process does not depend on this velocity so that it can be safely set to zero. The fake particle is then turned into a fake flight and passed on to the `score_sources` subroutine.

This subroutine works in exactly the same way as during run time. The distinction arises only in the subroutine calls that set up the “data” for each source type; in each case, the values of sampled parameters are replaced by their averages. For example, in the case of a plate source (subroutine `plate_data`), the parameter `wa = 3`; in run time, this is the square of the unscaled velocity vector obtained by sampling from a Gaussian distribution in each of the three spatial directions (via `rn_gauss_next`). The parameter `vpuff` serves this exact same purpose in subroutine `puff_data`. In this case, one also needs the average velocity vector as sampled from a cosine distribution, which is $(0, 0, 8/3\sqrt{6\pi})$. Note that this is correct only in the default case of a pure cosine distribution; the more general “cosine to an exponent” case is not handled. For a plate source with a specified energy distribution, subroutine `plt_e_bins_v_details` uses the input parameter `wa` (in call from `plate_data`) / `urd` (its argument list) as a switch, with a value of the macro `real_unused` telling it to compute the average energy from the user specified energy distribution.

For the volume source, the user specifies the source’s average velocity, $\vec{v}_{\text{src}}$, and temperature, T . The former becomes the fake particle’s velocity. Its energy is then just $1.5T + 0.5mv_{\text{src}}^2$.

Scoring the recombination source is more complex due to the associated light emission and electron energy losses and sinks. The fake particle’s velocity is set from the recombining ion’s flow velocity in `post_process_source_scores`. In subroutine `recombination_data`, the electron energy

source and loss rates are evaluated, as are the wavelength and emission rate of the diagnostic lines being simulated. The ion energy lost is the thermal value analogous to that shown above for the volume source. The flow velocity vector and magnitude are loaded into dedicated “Maxwellian emitter” slots in the `scoring_data` array. When `add_scores` is called at the end of `score_sources` and it in turn calls `handle_detectors`, these data are used to compute the corresponding full emission spectrum. This provides an enormous improvement in signal quality over a collision-based score, making this another very useful application of post-processed scoring.

5 Final Processing

The expressions in Sec. 3 above are incorporated into subroutine `stat_acc`, although their equivalence is difficult to discern due to the bookkeeping associated with the compression of the S arrays. This routine is called at various points during the run to first accumulate “flight” data into a “fragment” array, and then the “fragments” into the “final” array. The end result of this process is the `output_grp` array via the call to `stat_acc` in the “Merge results” macro invoked by subroutine `do_flights_master`.

As will be shown in more detail below, the mean tally values for each source group are scaled by the total current in that group, which is obtained by summing over the group’s $N_{\text{seg},i}$ segments:

$$\Gamma_i = \sum_{\ell=1}^{N_{\text{seg},i}} \Gamma_{i,\ell}. \quad (30)$$

In the steady state mode of operation, Γ_i represents a number of particles per second; in the time dependent mode, Γ_i is a number of particles. Obviously, these quantities are used in combining the $S_1^i(z)$ and $S_2^i(z)$ over source groups. But, they are also used to scale the user-specified relative statistical weights to get the ρ_j appearing in Eq. (1), etc. above and in setting the probabilities for sampling flights from the source segments.

Namely, the user can specify an arbitrary set of relative weights, $r_{i,\ell}$, for segments $\ell = 1 \rightarrow N_{\text{seg},i}$ in group i . A normalization factor is computed for each source group:

$$r_{\text{norm},i} = \left(\sum_{\ell=1}^{N_{\text{seg},i}} \Gamma_{i,\ell} / r_{i,\ell} \right) / \left(\sum_{\ell=1}^{N_{\text{seg},i}} \Gamma_{i,\ell} \right). \quad (31)$$

The cumulative probability distribution used in sampling flights in group i is, e.g., for segment ℓ :

$$p_{i,\ell} = \left(\sum_{m=1}^{\ell-1} \Gamma_{i,m} / r_{i,m} \right) / \left(\sum_{m=1}^{N_{\text{seg},i}} \Gamma_{i,m} / r_{i,m} \right). \quad (32)$$

These probabilities are computed without the weight normalization factor Eq. (31) since it would appear in both numerator and denominator. The actual sampling procedure is more complex and offers additional flexibility; see the `sources` class and `sourcetest`.

If flight j in group i is sampled from segment i , the relative weight is set to:

$$\rho_j = r_{i,\ell} \times r_{\text{norm},i}. \quad (33)$$

Scaling and final processing of the output arrays is carried out by a couple of loops and subroutine calls near the end of subroutine `do_flights_master`. In the first of the loops, the means and variances are scaled by the relative source currents:

$$P_1^i(z) = \frac{\Gamma_i}{\sum_i \Gamma_i} S_1^i(z) \quad (34)$$

and

$$P_2^i(z) = \left(\frac{\Gamma_i}{\sum_i \Gamma_i} \right)^2 \frac{S_2^i(z)}{\left(\sum_{j=1}^{N_i} \rho_j - 1 \right)}. \quad (35)$$

At this point, S represents `output_grp`, and P represents `out_post_grp`; the latter is also summed over source groups to obtain `output_all`.

A second set of loops performs a final scaling by the global total current, $\sum_i \Gamma_i$, and a division by the time interval for time dependent runs. The latter transforms the input “current” from a number of particles into an average number of particles per second; in the interest of clarity we do not explicitly show this factor here. At this stage, the variance is transformed into the relative standard deviation that is used externally to quantify the accuracy of the code’s results:

$$P_2^i = \frac{(P_2^i)^{1/2}}{P_1^1}, \quad (36)$$

and

$$P_1^i = \left(\sum_i \Gamma_i \right) S_1^i \quad (37)$$

Namely, the expected error in a statistical estimate of the mean μ via N samples is $\sigma/\sqrt{N}$, where σ is the standard deviation and σ^2 is the variance. The relative standard deviation “rsd” is then

$$\text{rsd} = \frac{\sigma}{\mu\sqrt{N}}. \quad (38)$$

Note that Eq. (5) and so on have $N \leftrightarrow \sum \rho_j$ in the denominator of the variance even though $N - 1$ appears in most textbook expressions. The “-1” accounts for the fact that the same sample is used to estimate the mean μ and the variance. While $N \gg 1$ in most DEGAS 2 runs, rendering the difference between N and $N - 1$ negligible, retaining $N - 1$ yields a maximum relative standard deviation of exactly unity when $\rho_j \equiv 1$ (a single score; see below), facilitating interpretation of results. In the process of computing the relative standard deviation, we effectively multiply the S_2 by $N/(N - 1)$. The $N - 1$ part of this is the denominator in Eq. (35); the N ends up being cancelled by the $1/\sqrt{N}$ in Eq. (38) and does not explicitly appear in Eq. (40).

The P_1^i and P_2^i quantities in Eqs. (40) and (39) again end up in the `out_post_grp` array and the sum over source groups in `output_all`.

For purposes of explication, we combine the expressions in Eqs. (34 - (39) into:

$$P_1^i = \Gamma_i S_1^i, \quad (39)$$

and

$$P_2^i = \frac{\left[\Gamma_i^2 S_2^i / \left(\sum_{j=1}^{N_i} \rho_j - 1 \right) \right]^{1/2}}{\Gamma_i S_1^i}. \quad (40)$$

Finally, post-processed tallies are added to the `out_post_grp` array and the “conversion” factors are applied via calls to subroutine `final_conversion`. The mean values in this array are summed over groups to obtain the means in `out_post_all`. Since the effects of the post-processing tallies on the variance are *not* accounted for, the rsd’s from `output_all` are copied into the “variance” entry in `out_post_all`.

6 External Use of Output Arrays

The principal output of the code is contained in the mean values of the `out_post_grp` and `out_post_all` arrays. Invariably, the relative standard deviations in these arrays are used to characterize the uncertainty in the results, although the inclusion of post-processed tallies may render those values inaccurate. To be safe, one should use the `rsd`'s from “test” tallies, which are not available for post-processed scoring, to quantify the uncertainty in a simulation.

None of the scaling factors and transformations described in Sec. 5 are ever applied to the `output_grp` array since it is used as input to restarted simulations.

This fact is forms the basis for the `acc_output` utility, which is used to effectively average over a number of simulations. A related application is to simulate neutral transport in the presence of a time varying plasma background (since the background is always assumed fixed in time during a given run of DEGAS 2).

The `acc_output` utility is called after each run with the name of a “base” output netCDF file specified as a command line argument. On the first such call, this file is assumed to not exist. The output file specified in the `degas2.in` file is read in and its `output_grp` array is scaled by Γ_i for the mean and Γ_i^2 for the variance. These source strengths are read in from the corresponding background file. The resulting array is written into the `output_grp` array in the “base” file. Other data appear in the “base” file at this point, although they are not used.

On subsequent calls, the `output_grp` data in the `degas2.in` output file are again read in and scaled by the source strengths. The latter come from the current background netCDF file; this is an important consideration in that they may differ from those of the previous run in a time dependent application. At this point, the code invokes expressions Eq. (8), (9), and (12), just as in the `stat_acc` routine; these results are placed in the `output_grp` routine of the “base” file, facilitating successive repetitions of this process.

With these data now representing a non-trivial product of the input data, expressions equivalent to Eqs. (39) and (40) are applied and inserted into `out_post_grp`; these are summed over groups to fill `output_all`. By design, the Γ_i factors have already been applied. Because the background plasma data may be different in the constituent simulations, post-processed tallies cannot be handled; the code will throw an assertion if any are present. The conversions performed by subroutine `final_conversions` are applied, yielding `out_post_grp` and `out_post_all`. Due to the exclusion of post-processed tallies, the variances of both sets of output will be accurate. The resulting “base” file can then be used just like a normal output file in conjunction with `outputbrowser`, `geomtesta`, etc. The two are not completely equivalent since `acc_output` scales the `output_grp` array and does not update the `output_2D_coupling` array (those data can be obtained directly from the constituent tallies via `outputbrowser`). The arrays `output_num_flights` and `output_random_seed` are not utilized, either. The total, accumulated weight is available in `output_weight_grp`.

7 A Single Score

In the special case of a single score ξ_x with weight ρ_x in a particular zone x , Eq. (1) simplifies to

$$S_1 = \frac{\rho_x \xi_x}{\rho_{\text{tot}}}, \quad (41)$$

with $\rho_{\text{tot}} \equiv \sum_j \rho_j$. Equation (5) becomes

$$S_2 = \frac{\rho_x \xi_x^2}{\rho_{\text{tot}}} - \frac{\rho_x^2 \xi_x^2}{\rho_{\text{tot}}^2} \quad (42)$$

$$= \xi_x^2 \frac{\rho_x}{\rho_{\text{tot}}} \left(\frac{\rho_{\text{tot}} - \rho_x}{\rho_{\text{tot}}} \right) \quad (43)$$

Then, the relative standard deviation, which is effectively $\sqrt{S_2}/[\sqrt{(\rho_{\text{tot}} - 1)S_1}]$ reduces to:

$$\text{rsd} = \frac{1}{\sqrt{\rho_x}} \left(\frac{1 - \rho_x/\rho_{\text{tot}}}{1 - 1/\rho_{\text{tot}}} \right)^{1/2}. \quad (44)$$

In the limit of $\rho_{\text{tot}} \gg 1, \rho_x$, the $\text{rsd} \rightarrow 1/\sqrt{\rho_x}$. In the usual case of uniform weighting, $\rho_x = 1$ and $\rho_{\text{tot}} \rightarrow N$, $\text{rsd} \equiv 1$, as noted above.

Note, however, that when performing importance sampling, one can obtain single scores having $\rho_x < 1$ and, thus, an $\text{rsd} > 1$. This is simply a reflection of that score carrying less than average statistical weight. The rsd values in regions of the problem space having adequate statistics can be interpreted via the same guidelines as simulations with uniform weighting (e.g., as in Sec. 2.4 of the User's Manual).

8 Handling of Relative Weights for Importance Sampling

The path of the relative weight factors through the code, particularly in the time dependent case, is sufficiently complex that it warrants exposition.

8.1 Setting up the source (e.g., in `defineback`)

First, the relative weights are stored in the array `source_segment_rel_wt`. The default value of 1 is set in subroutine `init_wt_alias` in `writeback.web`. Since importance sampling is rarely used, and schemes for employing it must be adapted carefully to the problem at hand, no specific code for it is provided; the user is responsible for inserting code that will assign the desired values.

As an example, we provide a rough description of its application to the time dependent version of the synthetic Gas Puff Imaging diagnostic, where the objective is to ensure radially uniform sampling of particles from the snapshot source. For simplicity, the relative weight computation is integrated into subroutine `set_snapshot_source`; the lines establishing the default values in subroutine `init_wt_alias` are then commented out. In `set_snapshot_source`, the problem space is divided up, somewhat arbitrarily, into 1 cm radial bins, and the weights of the snapshot particles are accumulated into them. The total weight in each bin is then divided by the global total to yield a fraction; this becomes the relative weight factor associated with each bin and, thus, each snapshot particle in that bin.

As in the default version of subroutine `set_snapshot_source`, the `source_current` for each particle (i.e., each particle is a "source segment") is set to the snapshot particle's weight (without the relative weighting factor) and the total current, `so_tot_curr` for the source group. The relative weight factors are then combined with the currents in Eq. (32) to set up the sampling distribution. The idea is that bins deeper into the plasma, where penetration is poor, will have smaller relative weights. This will cause them to be preferentially sampled, as in Eq. (32). An unbiased result is achieved by reducing the statistical weight of these particles in the main code by the same relative weight factor. The overall weight normalization factor, `so_wt_norm` is computed (by default) from the currents and relative weights via Eq. (31) in subroutine `set_prob_alias`.

8.2 Source sampling

As is noted in `sources.web`, subroutine `sample_sources`, the value of `source_segment_rel_wt`, scaled by `so_wt_norm`, is assigned *temporarily* to the sampled particle's weight, `pt_w`. In the `ffl_copy`

macro (`flight_frag.hweb`) invoked in `doflights.web`, subroutine `setup_flight_frag`, this scaled relative weight is transferred into the `ff_float_pt_w` entry in the `ff_particles_float` array.

The associated `ff_label_init` macro, also in `flight_frag.hweb`, returns this quantity as a macro parameter, `rel_wt`, and then sets the flight's (instantaneous) statistical weight, `pt_w`, to 1. The overall weight of the flight, `stat_wt_tot_flt`, is equated to that `rel_wt` parameter with the invocation of the macro in `flight.web`, subroutine `do_flights`; it is then globally accessible via the “stat” (sa) class and associated common blocks. This is the weight that is passed to subroutine `stat_acc` to accumulate all of the scores from this flight and to obtain the total sampled weight for the source group. That is, the individual scoring routines, such as `score_test`, record only `pt_w`. However, when all of those scores are combined with those from preceding flights, they are effectively scaled by the relative weight factor via `stat_wt_tot_flt`. Note that with importance sampling the total weight in the source group is only approximately equal to the number of flights sampled.

8.3 Snapshot score and PDF

Flights (in any source group) reaching the end of the time step in time dependent runs trigger a final call to `score_test` with the snapshot estimator, `tl_est_snapshot`. This will, obviously, lead to the accumulation of data needed for tallying the snapshot density, etc. But, it also results in a call to subroutine `build_snapshot_pdf` in `score.web`. The weight of the resulting entry in the snapshot particle distribution function, `sn_float_pt_w`, is recorded there as the product of `pt_w` and `stat_wt_tot_flt`. Once all of the particles in a source group have been tracked, the weights of the newly created snapshot particles are scaled by the total current in the group, `so_tot_curr` (and the rarely used, arbitrary factor `so_scale`), and divided by the total weight in the group, `output_weigh_grp`. As noted above, in the subsequent time step, subroutine `set_snapshot_source` (`writeback.web`) sets the `source_current` associated with this particle to the value of this `sn_float_pt_w` entry in the snapshot arrays.